# Claude Code 입문하기

🧠 **AI로 인한 변화들**

▼ **AI로 인한 구현 가치의 폭락**

- 과거와 현재 개발 트렌드의 변화
  - ASIS: 설계 → **개발** → 운영
  - TOBE: **설계** → 개발 → **운영**
- 구현 가치의 폭락
  - 작업의 열세에서 인간이 AI를 이길 수 없기 때문에, 적절한 아키텍처가 유지되는 한 인간은 더 이상 AI를 이길 수 없음
  - '설계가 잘 되었다면 구현은 신입사원도 충분하다'는 가설을 증명하고 있고, '적절한 아키텍처 상에서 수작업으로 코드를 양산해 온 소프트웨어 엔지니어'의 일자리가 AI에 뺏기는 것
  - 즉, 코드를 양산하는 개발자의 시장 가치는 폭락하는 시대

▼ **AI로 인한 개발자의 역할 변화**

- 프롬프트 엔지니어링(Prompt Engineering) → 컨텍스트 엔지니어링(Context Engineering)
  - 컨텍스트란 단순한 프롬프트 문장이 아니라, LLM이 답변을 생성하기 전 볼 수 있는 모든 정보(지침, 대화이력, 장기 메모리, 외부 정보, 가용 도구 등) 를 의미함
  - 에이전트의 성공과 실패는 이제 모델의 성능보다 컨텍스트의 질에 달려 있음
  - 컨텍스트 엔지니어링은 상황에 따른 동적인 시스템 설계로, 올바른 정보와 도구를 정확한 시점에 LLM에 제공하는 과정임

| 비교요소 | 프롬프트 엔지니어링 | 컨텍스트 엔지니어링 |
|---|---|---|
| 주요 작업 | 단순 텍스트 명령 | 맥락, 도구, 장기 메모리 등을 종합적으로 제공 |

| 비교요소 | 프롬프트 엔지니어링 | 컨텍스트 엔지니어링 |
|---|---|---|
| 정보의 질 | 제한적, 단편적 | 포괄적, 맥락적 |
| 사례 | 간단한 코드 생성 ("React 버튼 만들어 줘") | 전체 서비스 흐름을 파악한 코드 생성 (기존 코드 구조, 요구사항 등을 제공) |

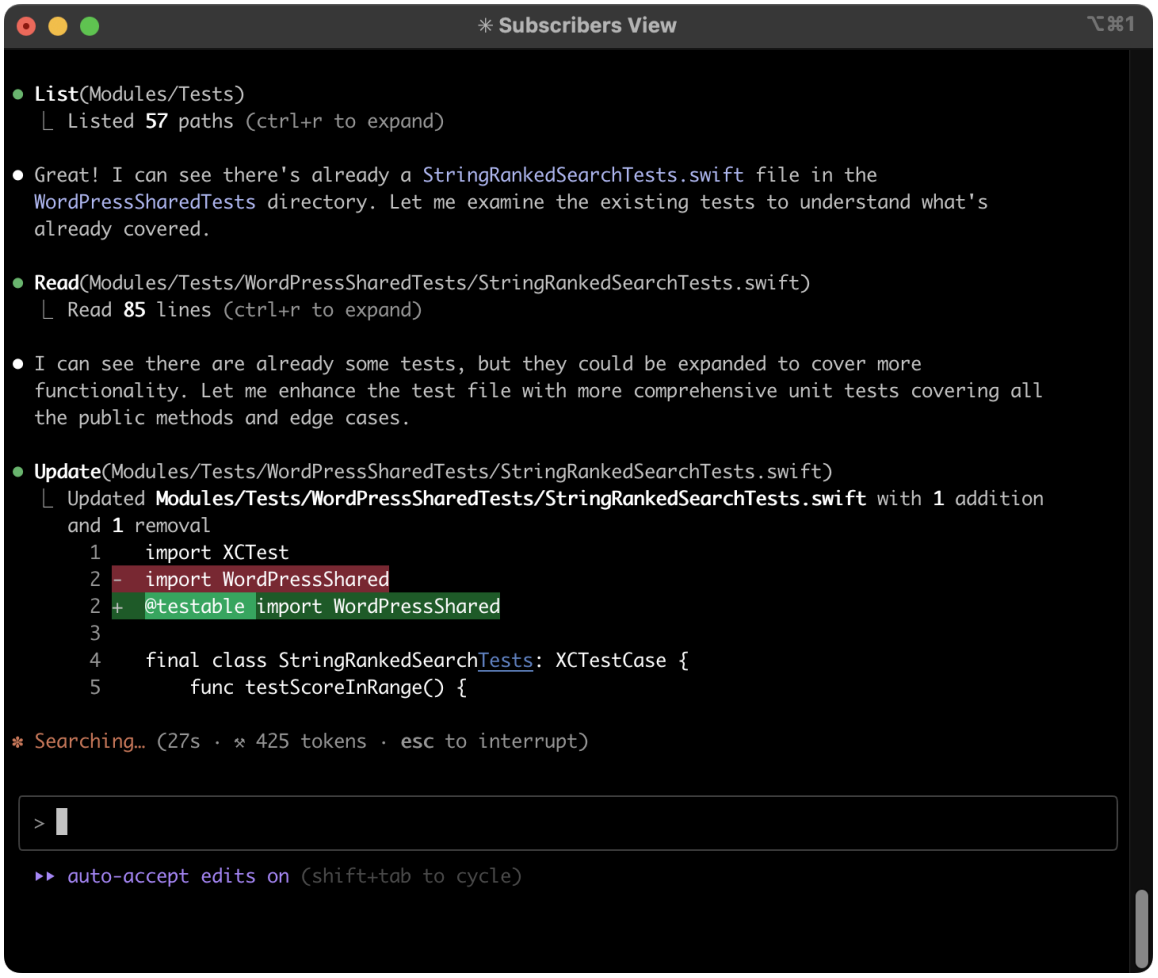
- 맥락 전달의 중요성
  - “내 기술의 90%는 0달러가 되었고, 나머지 10%는 1,000배의 가치를 얻게 됐다.” by Kent Beck
    - 0달러가 된 90%: 단순한 코드 작성 능력
    - 1,000배가 된 10%: 개발 의도를 어떻게 AI에게 잘 전달할 것인지에 대한 능력
  - 제품의 복잡도에 따른 차이
    - 단순한 제품: 대충 말해도 잘 알아들음
    - 복잡한 제품: 좋은 정보가 필요한 시대, 메타 데이터의 중요성 극대화

- AI로 인한 현실의 변화
  - 구현 가치가 폭락함에 따라 1인 창업이 주류가 되는 시대가 되어가고 있고, 개발은 AI에게 위임하되 비즈니스적 가치에 대한 고민과 제품에 대한 의사결정이 중요해지고 있음
  - 개발은 AI가 해주지만 비즈니스 규모가 증가함에 따른 대응 능력은 부족하기에, 기본기를 바탕으로 하는 문제해결 능력이 더욱 부각되고 있음

▼ **Claude code 입문하기**

# [ Claude Code란? ]

- CLI 기반의 AI 도구로, 터미널에서 실행되기에 기존 개발 환경과 함께 사용됨
- CLI 기반이기에 git이나 bash 등에도 제약 없이 접근이 가능하기에 널리 사용됨
- Claude Code는 로컬 환경에서만 동작하기에 보안적으로도 안전하며, 작업을 요청한 경우 프로젝트 전체를 탐색하며 작업을 진행함




# [ Claude Code 설치하고 실행하기 ]

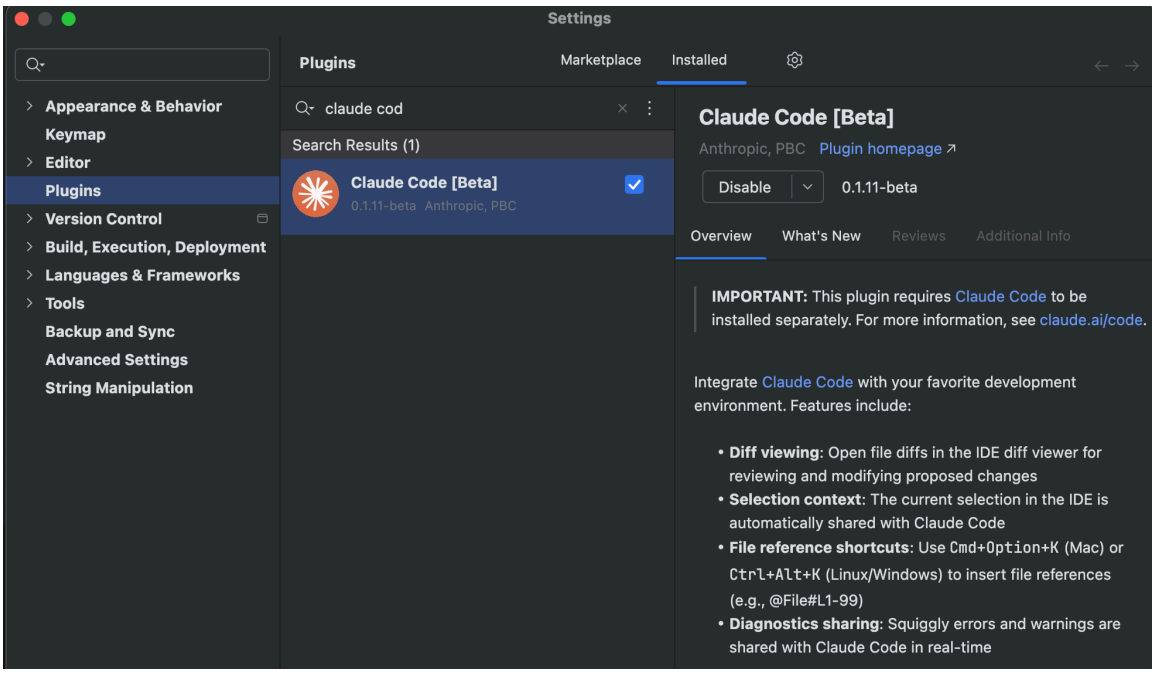
## Claude Code 설치하기

```
# Claude Code 설치
npm install -g @anthropic-ai/claude-code

# 프로젝트 디렉토리로 이동
cd your-project

## Claude Code 실행
claude
```

## Claude Code 플러그인 설치하기

- Claude Code는 다양한 개발 환경에서 플러그인을 제공하기에 플러그인을 설치해 주면 좋음

- Diff 보는 화면, 컨텍스트 공유 등과 같은 기능을 통해 사용성을 높일 수 있음

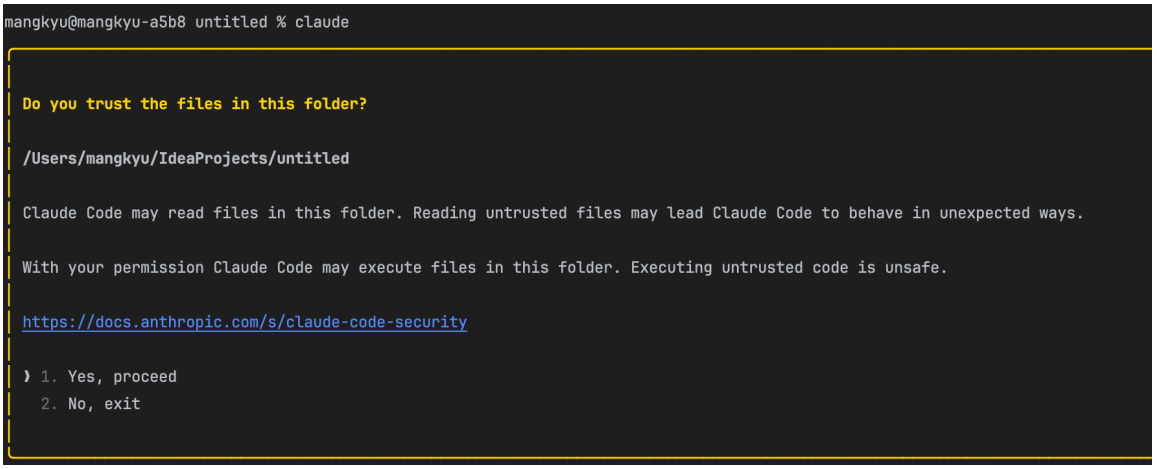



## Claude Code 실행하기

- 터미널에 claude를 입력하면 Claude Code를 실행시킬 수 있음
- IntelliJ를 사용중이라면 플러그인 설치 후에 Command + Esc 키로 손쉽게 실행시킬 수 있음

```
claude
```

- 이후에 다음과 같은 창이 나오면, proceed를 눌러주면 됨




## init 명령어

- 프로젝트에서 Claude Code를 처음 사용하고자 할 때, init 명령어를 호출해주면 좋음
- 그러면 프로젝트 폴더에 CLAUDE.MD 파일이 생성되고 그 안에 프로젝트 관련 정보가 담겨짐
- 이를 통해 작업 시에 많은 컨텍스트를 얻으면서 더 좋은 성능을 보일 수 있음

```
/init
```

- 만약 CLAUDE.MD 파일을 홈 디렉토리에 직접 생성하고 작성할 경우, 모든 프로젝트에 공통되는 규칙들을 정의할 수도 있음

```
~/.claude/CLAUDE.MD
```

- init 명령어를 실행하면 Claude Code가 프로젝트를 분석하기 시작함

```
● I'll analyze the codebase to understand its structure and create a comprehensive CLAUDE.md file.

● Update Todos
  └ ☐ Explore project structure and identify project type
    ☐ Analyze build configuration and development scripts
    ☐ Understand code architecture and main components
    ☐ Check for existing documentation and configuration files
    ☐ Create comprehensive CLAUDE.md file

● Update Todos
  └ ☐ Explore project structure and identify project type
    ☐ Analyze build configuration and development scripts
    ☐ Understand code architecture and main components
    ☐ Check for existing documentation and configuration files
    ☐ Create comprehensive CLAUDE.md file

● List(.)
  └ Listed 121 paths (ctrl+r to expand)

● Read(build.gradle.kts)
  └ Read 55 lines (ctrl+r to expand)

● Read(settings.gradle.kts)
  └ Read 11 lines (ctrl+r to expand)
```

- 분석이 완료되고, 실제 CLAUDE.MD와 같은 파일을 생성하는 작업을 진행하게 될 때에는 권한을 물어보게 됨

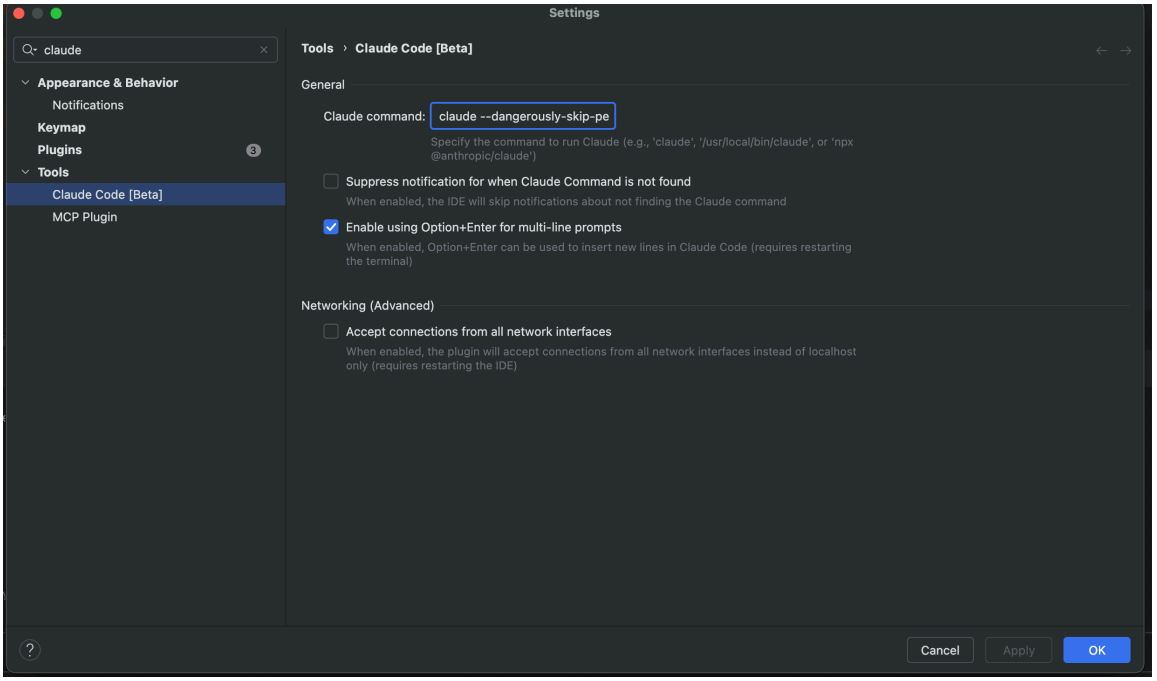
- 상황에 맞게 필요한 옵션을 선택해주면 됨
  - Yes: 제안된 변경사항을 CLAUDE.md에 적용하고, 이후에도 물어봄
  - Yes, and don't ask again this session: 이번 세션 동안 모든 변경사항을 자동 적용함
  - No, and tell Claude what to do differently: 변경을 적용하지 않고 Claude에게 피드백을 줌

```
Opened changes in IntelliJ IDEA

Do you want to make this edit to CLAUDE.md?
❯ 1. Yes
  2. Yes, and don't ask again this session (shift+tab)
  3. No, and tell Claude what to do differently (esc)
```

- 매번 권한을 설정하기에 번거롭다면, claude 실행 시에 --dangerously-skip-permissions 옵션을 줄 수 있음
- 해당 옵션은 실행 시에 권한 확인을 생략함

```
claude --dangerously-skip-permissions
```

- 만약 IntelliJ 플러그인을 통해 claude를 위의 옵션과 함께 실행시키고자 할때는 플러그인 설정을 수정해주면 됨




# [ Claude Code 사용팁 ]

## clear 명령어

- 이전과 다른 새로운 작업을 시작할 때에는 항상 clear를 통해 현재의 대화 내역을 초기화해주는 것이 좋음
- 성능을 최적화하고 관련 없는 정보를 제거하여, Claude의 집중력이 흐트러지는 것을 방지할 수 있음

```
/clear
```

## @로 맥락 전달하기

- 명령을 작성할 때 @를 사용하면 컨텍스트를 전달할 수 있음

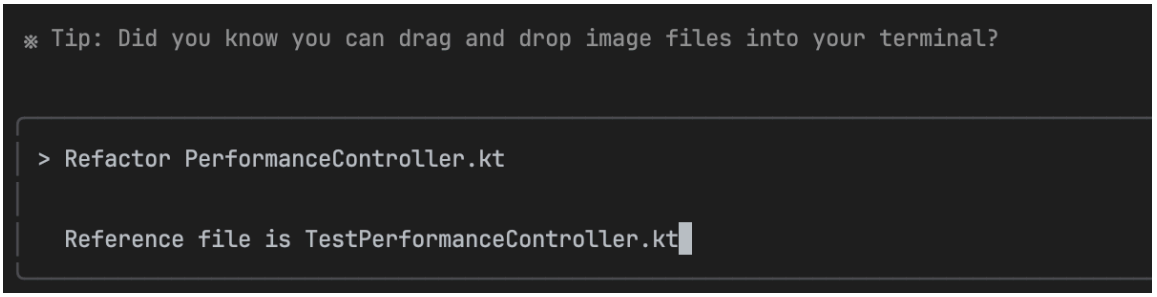
## 연속적인 여러 작업들 요청하기

- 여러 건의 작업을 연속적으로 처리해주어야 하는 경우에는 여러 명령들을 입력시켜 두면 됨
- Claude Code는 처리해야 하는 요청들을 쌓아두고, 현재 요청의 처리가 완료되면 이어서 해당 요청을 처리함

```
analyze current project
optimize performance by code refactoring
run and fix failed tests
```

## 명령 작성 시에 사용팁

- 명령 작성 시에 개행이 필요한 경우에는 Shift + Enter(안되면 /terminal-setup) 혹은 Ctrl + J를 사용하면 됨




- 명령어에 클립보드의 이미지를 넣고 싶다면, Ctrl + V로 넣을 수 있음

## #으로 작업 기억하기

- 이전에 수행했던 작업 내역을 기록해두면, 반복적으로 Claude Code에 동일한 명령을 내릴 필요가 없어짐
- #을 누르면, 이전 작업을 CLAUDE.MD에 추가하여 기억시킬 수 있음

```
#
```

## 기타

- 이전 채팅을 재활용하고 싶은 경우, 위 화살표 누르면 이전 채팅으로 갈 수 있음
- Esc를 두 번 누르면 모든 내역을 볼 수 있음

## [ 요약 및 정리 ]

- IDE가 제공하는 터미널 환경에서 Claude Code 플러그인과 함께 사용하기
- Claude Code를 --dangerously-skip-permissions 옵션과 함께 실행하기
- 새로은 프로젝트에 Claude Code를 처음 실행할 때 /init 해주기
- 새로운 작업을 시작하기 전에는 항상 /clear 하기
- 작업을 여러 건 해야하는 경우, 요청을 넣어둬서 연속적으로 이어서 시키기
- 개행을 위해서는 Shift + Enter(안되면 /terminal-setup) 혹은 Ctrl + J를 사용하기
- 컨텍스트를 불러올 경우에는 @를 사용하면 됨
- 클립보드의 이미지를 넣으려면 Ctrl + V로 넣을 수 있음
- 작업한 내역을 기록해야 할 경우에는 #으로 기억시키기
- 위 화살표 누르면 이전 챗으로 갈 수 있고, Esc 두 번 누르면 모든 메시지 볼 수 있음

## [ Custom Command(커스텀 명령어) 생성하기 ]

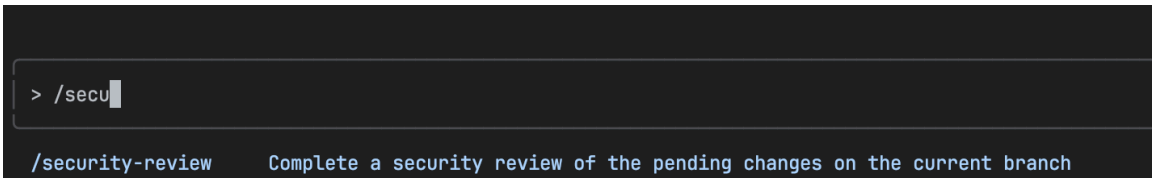
- 반복적으로 수행중인 작업들을 프롬프트로 작성해두고 Claude Code에게 시키면 자동화할 수 있음
- Claude Code는 이를 위한 커스텀 명령어를 제공함
- 커스텀 명령어를 Claude Code 전체 명령어로 설정할지 혹은 특정 프로젝트에서만 사용할지에 따라 먼저 폴더를 생성해주어야 함

```
// 특정 프로젝트에 종속적인 Slash 명령어 위치
mkdir -p .claude/commands

// 전역적인 Slash 명령어 위치
mkdir -p ~/.claude/commands
```

- 이후 해당 폴더 하위에 마크다운 파일을 추가해줌

```
echo "Review this code for security vulnerabilities, focusing on:" >
~/.claude/commands/security-review.md
```

- 이제 다음과 같이 파일 이름과 동일한 명령어를 Claude Code에서 실행할 수 있음
- 커스텀 커맨드에 인자가 필요한 경우, $ARGUMENTS를 활용해 입력받을 수도 있음




# [ Claude Code Subagents ]

- 특정한 작업을 위한 워크플로와 향상된 컨텍스트 관리를 위한 서브 에이전트 기능이 제공됨
- 서브 에이전트를 위한 맞춤형 시스템 프롬프트 등을 제공하여 효율적으로 문제 해결을 할 수 있음
- 예를 들어 주어진 코드를 문제없이 개선시키는 리팩토링의 전문가가 필요한 경우에, 우리는 리팩토링을 전문으로 하는 서브 에이전트를 만들어 사용할 수 있음
- 따라서 서브에이전트는 Claude Code가 작업을 위임할 수 있는 사전 구성된 AI 인격체로 이해되며, 각 서브에이전트는 다음과 같은 특징을 지님
    - 특정한 목적과 전문 영역이 있음
    - 기본 대화와 별도로 자체 컨텍스트 창을 사용함
    - 사용하도록 허용된 특정 도구로 구성 가능
    - 행동을 안내하는 사용자 지정 시스템 프롬프트를 지님

# [ SuperClaude ]

## SuperClaude란?

- 우리가 직접 서브 에이전트를 매번 만들고 하는 것은 번거롭기에, 잘 만들어진 Super Claude를 활용하는 것이 바람직함
- SuperClaude는 19개의 특수 명령어와 9개의 페르소나를 통해 Claude Code의 활용도를 극대화해줌
- SuperClaude를 사용하면, 자동화된 커밋 메시지, 코드 리뷰, 정돈된 changelogs, 문서화 등의 기능을 사용할 수 있음
- 하지만 토큰의 사용량이 훨씬 많아지기에 주의할 필요가 있다.

## SuperClaude 설치하기

```
git clone https://github.com/SuperClaude-Org/SuperClaude_Framework
cd SuperClaude
./install.sh
```

## SuperClaude의 다양한 명령어들

- /sc:analyze: 코드 분석
- /sc:build: 빌드
- /sc:implement: 기능 구현
- /sc:improve: 개선
- /sc:troubleshoot: 문제 해결
- /sc:test: 테스트 실행
- /sc:document: 문서 작성
- /sc:git: 깃 사용
- /sc:design: 시스템 디자인
- /sc:explain: 설명하기
- /sc:cleanup: 코드 정리

**Command: Implementation**

- 특정한 기능을 구현하는 데 활용할 수 있으며, SuperClaude는 구현 내용에 따라 적합한 전문가(프론트엔드, 백엔드, 보안)와 도구를 자동으로 활성화함
- implementation에는 다음과 같은 옵션들이 존재함
  - `-type component|api|service|feature|module` : 구현 종류
  - `-framework react|vue|express|django|etc` : 대상 프레임워크
  - `-safe` : 대화형 접근 방식으로 구현
  - `-iterative` : 점진적 단계로 검증하며 개발함
  - `-with-tests` : 테스트도 함께 구현
  - `-documentation` : 코드와 함께 문서 작성
- 권장되는 사용법은 다음과 같음
  - type을 명시하기(component vs service vs feature)
  - framework로 특정 기술 스택을 명시하라
  - 프로덕션 코드를 위해 safe를 사용하고, 복잡한 기능이라면 iterative를 사용하라

**Command: Improve**

- 작성된 코드를 리팩토링하거나, 성능을 개선하는 등의 경우에 활용할 수 있음
- improve에는 다음과 같은 옵션들이 존재함
  - `-type quality|performance|maintainability|style` : 초점을 두는 작업
  - `-safe` : 위험도가 낮은 변경들만 반영함
  - `-preview` : 변경될 내역들을 보여줌
- 권장되는 사용법은 다음과 같음
  - 항상 preview를 먼저 사용해서 변경되는 부분을 확인하기
  - 안정성이 필요하다면 safe를 통해 위험도 높은 리팩토링은 방지하기
  - 전체 코드베이스보다는 작은 파일 혹은 모듈 단위의 작업에 뛰어남

그 외에도 각각의 기능에 대한 보다 자세한 사용법은 공식 문서를 통해 참고해주도록 하자.